

Sumário

1	Templates	1	6	Matemática	10
1.1	Código padrão (template)	1	6.1	Fórmulas	10
1.2	Init script	1	6.2	MDC & MMC	13
1.3	Compile script	1	6.3	Exponenciação	
			6.4	Binária	13
2	Fluxo e Emparelhamento	1	6.5	Números de Armstrong	13
2.1	Ford-Fulkerson	1	6.6	Primos	14
2.2	Dinic (fluxo máximo)	2	6.7	Crivo de Eratóstenes	14
			6.8	Fatoração Prima	14
3	Estruturas de Dados	2	6.9	Função Totiente de Euler (ϕ)	14
3.1	Coordinate Compression	2	6.10	Problema de Josefo (Josephus)	15
3.2	PBDS	3		Teorema Chinês do Resto (CRT)	15
3.3	Fenwick Tree (BIT)	3	7	Strings	15
3.4	BIT 2D	3	7.1	Suffix Array	15
3.5	Union-Find (DSU)	3	7.2	KMP	16
3.6	Tabela Esparsa (RMQ em $O(1)$)	3	7.3	Rabin-Karp	16
3.7	Segment Tree	4	7.4	Trie	17
3.8	Lazy Segment Tree	4	7.5	Função Z	17
3.9	Segment Tree dinâmica	5	7.6	Manacher	17
3.10	Segment Tree 2D dinâmica	5	8	Árvores	18
4	Grafos	6	8.1	LCA (Menor Ancestral Comum)	18
4.1	Ordenação Topológica	6	8.2	Centroid Decomposition	18
4.2	Dijkstra	6			
4.3	Bellman-Ford	6	9	Geometria	18
4.4	SPFA	7	9.1	Operações básicas	18
4.5	Floyd-Warshall	7	9.2	Fecho convexo	19
4.6	Caminho Euleriano	8	10	Algoritmos de Ordenação	19
4.7	Componentes Fortemente Conexas (SCC)	8	10.1	Insertion Sort	19
4.8	Árvore Geradora Mínima (MST)	9	10.2	Merge Sort	19
			10.3	Quicksort	20
5	Programação Dinâmica	9	10.4	Counting Sort	20
5.1	Edit Distance	9			
5.2	Problema da Mochila	9	11	Problemas Especiais	20
5.3	Longest Increasing Subsequence (LIS)	10	11.1	Torre de Hanoi	20

1 Templates

1.1 Código padrão (template)

```
#include <bits/stdc++.h>
using namespace std;

#define dbg(...) __f(__VA_ARGS__, __VA_ARGS__)
#define endl '\n'

template <typename Arg1>
void __f(const char* name, Arg1&& arg1) {
    cout << name << " : " << arg1 << endl;
}

template <typename Arg1, typename... Args>
void __f(const char* names, Arg1&& arg1, Args&&... args) {
    const char* comma = strchr(names + 1, ',');
    cout.write(names, comma - names) << " : " << arg1
        << " / ";
    __f(comma + 1, args...);
}

void solve() {
    // code here
}

int32_t main() {
    ios_base::sync_with_stdio(0); cin.tie(0);
    clock_t z = clock();

    int t = 1;
    // cin >> t;
    while (t--) solve();

    cerr << "Run Time : " << ((double)(clock() - z) /
        CLOCKS_PER_SEC) << "s" << endl;

    return 0;
}
```

1.2 Init script

```
for f in {A..}; do
    cat base.cpp >> "$f.cpp"
done
```

1.3 Compile script

```
src="$1"
g++ -O2 -Wall -Wextra -Wsign-compare -Wconversion -std=c++17 -g "$src"
```

2 Fluxo e Emparelhamento

2.1 Ford-Fulkerson

```
// Ford-Fulkerson (BFS = Edmonds-Karp)
// Retorna o fluxo maximo entre s e t
//  $O(V * E^2)$ 

const int INF = 1e18;
vector<vector<int>>> cap;
vector<vector<int>>> adj;
int n, m;

int bfs(int s, int t, vector<int>& pai) {
    fill(pai.begin(), pai.end(), -1);
    pai[s] = -2;
    queue<pair<int, int>> q;
    q.push({s, INF});

    while (!q.empty()) {
        auto [u, fluxo] = q.front();
        q.pop();

        for (auto& v : adj[u]) {
            if (pai[v] == -1 && cap[u][v] > 0) {
                pai[v] = u;
                int novo_fluxo = min(fluxo, cap[u][v]);
                if (v == t) return novo_fluxo;
            }
        }
    }
}
```

```

        q.push({v, novo_fluxo});
    }
}

return 0;
}

int ford_fulkerson(int s, int t) {
    int fluxo = 0;
    vector<int> pai(n + 1);
    int novo_fluxo;

    while (novo_fluxo = bfs(s, t, pai)) {
        fluxo += novo_fluxo;
        int u = t;
        while (u != s) {
            int p = pai[u];
            cap[p][u] -= novo_fluxo;
            cap[u][p] += novo_fluxo;
            u = p;
        }
    }

    return fluxo;
}

// no main:
adj.assign(n + 1, {});
cap.assign(n + 1, vector<int>(n + 1));
// para cada aresta a-b com peso c:
// adj[a].push_back(b); adj[b].push_back(a); cap[a][b]
// += c;

```

2.2 Dinic (fluxo máximo)

```

// O(V^2 * E) geral, O(E * sqrt(V)) para matching
bipartido
struct Dinic {
    struct Edge {
        int to, rev;
        int cap, flow;
    };

    int n;
    vector<vector<Edge>> g;
    vector<int> level, ptr;

    Dinic(int n) : n(n), g(n), level(n), ptr(n) {}

    void add_edge(int u, int v, int cap) {
        g[u].push_back({v, (int)g[v].size(), cap, 0});
        g[v].push_back({u, (int)g[u].size() - 1, 0, 0});
    }

    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        level[s] = 0;
        q.push(s);

        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (auto &e : g[u]) {
                if (level[e.to] == -1 && e.flow < e.cap) {
                    level[e.to] = level[u] + 1;
                    q.push(e.to);
                }
            }
        }
        return level[t] != -1;
    }

    int dfs(int u, int t, int f) {
        if (u == t || f == 0) return f;

        for (int &i = ptr[u]; i < g[u].size(); i++) {
            Edge &e = g[u][i];
            if (level[e.to] == level[u] + 1) {
                int pushed = dfs(e.to, t, min(f, e.cap
                    - e.flow));
            }
        }
    }
}

```

```

        if (pushed > 0) {
            e.flow += pushed;
            g[e.to][e.rev].flow -= pushed;
            return pushed;
        }
    }
}

return 0;
}

int max_flow(int s, int t) {
    int flow = 0;
    while (bfs(s, t)) {
        fill(ptr.begin(), ptr.end(), 0);
        while (int pushed = dfs(s, t, 1e18))
            flow += pushed;
    }
    return flow;
}

// Arestas do corte mínimo: visitáveis de s no
// grafo residual
vector<pair<int, int>> min_cut(int s) {
    vector<bool> vis(n);
    queue<int> q;
    q.push(s); vis[s] = true;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (auto &e : g[u])
            if (!vis[e.to] && e.flow < e.cap) {
                vis[e.to] = true;
                q.push(e.to);
            }
    }
    vector<pair<int, int>> cut;
    for (int u = 0; u < n; u++)
        if (vis[u])
            for (auto &e : g[u])
                if (!vis[e.to] && e.cap > 0)
                    cut.push_back({u, e.to});

    return cut;
}
};

```

3 Estruturas de Dados

3.1 Coordinate Compression

```

// build: O(N Log N) | get: O(Log N)

// === 1D ===
struct Compress {
    vector<int> c;
    void add(int x) {
        c.push_back(x);
    }
    void build() {
        sort(c.begin(), c.end());
        c.erase(unique(c.begin(), c.end()), c.end());
    }
    int get(int x) {
        return lower_bound(c.begin(), c.end(), x) - c.
            begin();
    }
    int size() {
        return c.size();
    }
};

// === 2D ===
struct Compress2D {
    vector<int> cx, cy;
    void add(int x, int y) {
        cx.push_back(x);
        cy.push_back(y);
    }
    void build() {
        sort(cx.begin(), cx.end());
        cx.erase(unique(cx.begin(), cx.end()), cx.end());
    }
    void build2() {
        sort(cy.begin(), cy.end());
        cy.erase(unique(cy.begin(), cy.end()), cy.end());
    }
};

```

```

    }
    int get_x(int x) {
        return lower_bound(cx.begin(), cx.end(), x) -
            cx.begin();
    }
    int get_y(int y) {
        return lower_bound(cy.begin(), cy.end(), y) -
            cy.begin();
    }
    int size_x() {
        return cx.size();
    }
    int size_y() {
        return cy.size();
    }
};

```

3.2 PBDS

```

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#define ordered_set tree<int, null_type, less<int>,
    rb_tree_tag, tree_order_statistics_node_update>
using namespace __gnu_pbds;

// ordered_set s;
// s.insert(10); s.insert(20); s.insert(30);
// O(logN):
// s.find_by_order(1); // Retorna iterador para o 20 (s[i])
// s.order_of_key(30); // Retorna 2 (posição do valor X no set)

```

3.3 Fenwick Tree (BIT)

```

// O(LogN)
struct FenwickTree {
    vector<int> v;
    int n;

    FenwickTree(int n) : n(n), v(n + 1, 0) {}

    FenwickTree(vector<int> const& v) : FenwickTree(v.size()) {
        for (int i = 0; i < (int)v.size(); i++) add(i + 1, v[i]);
    }

    // Soma prefixo [1, r]
    int sum(int r) {
        int ret = 0;
        for (; r > 0; r -= r & (-r)) ret += v[r];
        return ret;
    }

    // Soma intervalo [l, r] (1-based)
    int sum(int l, int r) {
        return sum(r) - sum(l - 1);
    }

    // Adiciona delta no índice idx (1-based)
    void add(int idx, int delta) {
        for (; idx <= n; idx += idx & (-idx)) v[idx] += delta;
    }
};

```

3.4 BIT 2D

```

// O(LogN * LogM)
struct FenwickTree2D {
    vector<vector<int>> bit;
    int n, m;

    FenwickTree2D(int n, int m) : n(n), m(m), bit(n + 1, vector<int>(m + 1, 0)) {}

    // Adiciona delta em (x, y) (1-based)
    void add(int x, int y, int delta) {
        for (int i = x; i <= n; i += i & (-i))
            for (int j = y; j <= m; j += j & (-j))

```

```

                bit[i][j] += delta;
            }

    // Soma prefixo [1,1] até (x, y)
    int sum(int x, int y) {
        int ret = 0;
        for (int i = x; i > 0; i -= i & (-i))
            for (int j = y; j > 0; j -= j & (-j))
                ret += bit[i][j];
        return ret;
    }

    // Soma retângulo (x1,y1) até (x2,y2) (1-based)
    int soma(int x1, int y1, int x2, int y2) {
        return sum(x2, y2)
            - sum(x1 - 1, y2)
            - sum(x2, y1 - 1)
            + sum(x1 - 1, y1 - 1);
    }
};

```

3.5 Union-Find (DSU)

```

// Oα(N) ≈ O(1)

vector<int> pai, tam;
void init(int n) {
    pai.resize(n + 1);
    tam.assign(n + 1, 1);
    iota(pai.begin(), pai.end(), 0); // pai[i] = i
}

int find(int x) {
    if (x == pai[x]) return x;
    return pai[x] = find(pai[x]); // path compression
}

// Union by Size: anexa a árvore menor na maior
void join(int x, int y) {
    x = find(x), y = find(y);
    if (x == y) return;
    if (tam[x] < tam[y]) swap(x, y);

    pai[y] = x;
    tam[x] += tam[y];
}

// Verifica se dois elementos estão no mesmo conjunto
bool mesmoConjunto(int x, int y) {
    return find(x) == find(y);
}

// Retorna o tamanho do componente que contém x
int tamanho(int x) {
    return tam[find(x)];
}

```

3.6 Tabela Esparsa (RMQ em O(1))

```

// Build: O(N * LogN) | Query: O(1)
// Suporta operações idempotentes: min, max, gcd, lcm

vector<int> log2;
vector<vector<int>> st;

void build_sparse(vector<int> &v) {
    int n = v.size();
    log2.resize(n + 1);
    log2[1] = 0;
    for (int i = 2; i <= n; i++) log2[i] = log2[i / 2] + 1;

    int K = log2[n] + 1;
    st.assign(K, vector<int>(n));
    st[0] = v;

    for (int j = 1; j < K; j++)
        for (int i = 0; i + (1 << j) <= n; i++)
            st[j][i] = min(st[j - 1][i], st[j - 1][i + (1 << (j - 1))]);
}

```

```
// Retorna mínimo no intervalo [L, R] (0-indexed,
// inclusivo)
int query_min(int L, int R) {
    int j = log2[R - L + 1];
    return min(st[j][L], st[j][R - (1 << j) + 1]);
}

// Versão para vetor 2D: mínimo em submatriz
// Build: O(N * M * LogN * LogM) | Query: O(1)
vector<vector<vector<vector<int>>>> st2d;

void build_sparse2d(vector<vector<int>>> &mat) {
    int n = mat.size(), m = mat[0].size();
    int K = log2[n] + 1, L = log2[m] + 1;
    st2d.assign(K, vector<vector<vector<int>>>>(L, vector<
        vector<int>>>(n, vector<int>>(m))));

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            st2d[0][0][i][j] = mat[i][j];

    for (int j = 1; j < L; j++)
        for (int i = 0; i < n; i++)
            for (int k = 0; k + (1 << j) <= m; k++)
                st2d[0][j][i][k] = min(st2d[0][j - 1][i][k],
                    st2d[0][j - 1][i][k + (1 << (j - 1))]);

    for (int i = 1; i < K; i++)
        for (int j = 0; j < L; j++)
            for (int k = 0; k + (1 << i) <= n; k++)
                for (int l = 0; l + (1 << j) <= m; l++)
                    st2d[i][j][k][l] = min(st2d[i - 1][j][k][l],
                        st2d[i - 1][j][k + (1 << (i - 1))][l]);
}

int query_min2d(int x1, int y1, int x2, int y2) {
    int i = log2[x2 - x1 + 1], j = log2[y2 - y1 + 1];
    return min({st2d[i][j][x1][y1],
        st2d[i][j][x1][y2 - (1 << j) + 1],
        st2d[i][j][x2 - (1 << i) + 1][y1],
        st2d[i][j][x2 - (1 << i) + 1][y2 - (1 <<
            j) + 1]});
}
```

3.7 Segment Tree

```
// O(N) build, O(logN) update e query
//
// Uso:
// SegTree st(n);
// for (int i = 1; i <= n; i++) st.v[i] = valor;
// st.build(1, 1, n);
// st.update(1, 1, n, pos, novoValor); // atualiza
// posição pos
// st.query(1, 1, n, L, r); // soma em [L, r]
//
// Indexação 1-based. Nó 1 = raiz.
// Filho esquerdo de no = 2*no, direito = 2*no+1
// tree[4*N] garante espaço suficiente para a árvore

struct SegTree {
    vector<int> v, tree;
    int n;

    SegTree(int n) : n(n) {
        tree.resize(4 * n);
        v.resize(n + 1);
    }

    int join(int a, int b) {
        return a + b;
    }

    // Constrói a árvore a partir de v[]
    // Chame: build(1, 1, n)
    void build(int no, int l, int r) {
        if (l == r) {
            tree[no] = v[l];
            return;
        }
        int m = l + (r - l) / 2;

```

```
        build(2 * no, l, m);
        build(2 * no + 1, m + 1, r);

        tree[no] = join(tree[2 * no], tree[2 * no + 1])
            ; // join (tipo da seg)
    }

    // Atualiza posição pos com valor val (substitui)
    // Chame: update(1, 1, n, pos, val)
    void update(int no, int l, int r, int pos, int val)
    {
        if (l == r) {
            tree[no] = val;
            return;
        }
        int m = l + (r - l) / 2;
        if (pos <= m) update(2 * no, l, m, pos, val);
        else update(2 * no + 1, m + 1, r, pos, val);

        tree[no] = join(tree[2 * no], tree[2 * no + 1])
            ; // atualiza nós pai
    }

    // Retorna soma em [ql, qr]
    // Chame: query(1, 1, n, ql, qr)
    int query(int no, int l, int r, int ql, int qr) {
        if (l > qr || r < ql) return 0; // fora do
            intervalo
        if (l >= ql && r <= qr) return tree[no]; //
            totalmente dentro
        int m = l + (r - l) / 2;

        return query(2 * no, l, m, ql, qr) + query(2 *
            no + 1, m + 1, r, ql, qr);
    }
};
```

3.8 Lazy Segment Tree

```
// O(N) build, O(logN) update e query
//
// Lazy: adia updates para filhos até que sejam
// necessários
// Sem lazy, update em intervalo seria O(N * LogN)
//
// Uso:
// SegTreeLazy st(n);
// for (int i = 1; i <= n; i++) st.v[i] = valor;
// st.build(1, 1, n);
// st.update(1, 1, n, L, r, val); // soma val em todos
// de [L, r]
// st.query(1, 1, n, L, r); // soma em [L, r]

struct SegTreeLazy {
    vector<int> tree, lazy, v;
    int n;

    SegTreeLazy(int n) : n(n) {
        tree.resize(4 * n, 0);
        lazy.resize(4 * n, 0);
        v.resize(n + 1, 0);
    }

    void build(int no, int l, int r) {
        if (l == r) {
            tree[no] = v[l];
            return;
        }
        int m = l + (r - l) / 2;
        build(2 * no, l, m);
        build(2 * no + 1, m + 1, r);
        tree[no] = tree[2 * no] + tree[2 * no + 1];
    }

    // Desce o lazy do nó atual para seus filhos
    // Chamado antes de qualquer descida na árvore
    void pushDown(int no, int l, int r) {
        if (lazy[no] == 0) return;
        int m = l + (r - l) / 2;
        // Aplica lazy nos filhos
        tree[2 * no] += lazy[no] * (m - l + 1);
        tree[2 * no + 1] += lazy[no] * (r - m);
    }
};
```

```

    lazy[2 * no] += lazy[no];
    lazy[2 * no + 1] += lazy[no];
    lazy[no] = 0; // limpa lazy do nó atual
}

// Soma val em todos os elementos de [ql, qr]
// Chame: update(1, 1, n, ql, qr, val)
void update(int no, int l, int r, int ql, int qr,
            int val) {
    if (l > qr || r < ql) return;
    if (l >= ql && r <= qr) {
        // Intervalo totalmente coberto: aplica
        // lazy aqui e para
        tree[no] += val * (r - l + 1);
        lazy[no] += val;
        return;
    }
    pushDown(no, l, r); // desce lazy antes de
        dividir
    int m = l + (r - l) / 2;
    update(2 * no, l, m, ql, qr, val);
    update(2 * no + 1, m + 1, r, ql, qr, val);
    tree[no] = tree[2 * no] + tree[2 * no + 1];
}

// Soma em [ql, qr]
// Chame: query(1, 1, n, ql, qr)
int query(int no, int l, int r, int ql, int qr) {
    if (l > qr || r < ql) return 0;
    if (l >= ql && r <= qr) return tree[no];
    pushDown(no, l, r); // desce lazy antes de
        dividir
    int m = l + (r - l) / 2;
    return query(2 * no, l, m, ql, qr) + query(2 *
        no + 1, m + 1, r, ql, qr);
}
};

```

3.9 Segment Tree dinâmica

```

// Usa ponteiros em vez de array estático
// O(logN) por operação, O(Q logN) espaço total (Q =
// número de updates)
//
// Quando usar no lugar da SegTree estática:
// - N muito grande (até 1e18) mas poucas operações
// - Coordenadas não compactadas (ex: queries em [1, 1
// e9])
// - Quer evitar alocar 4*N de antemão
//
// Uso:
// DinamicSegTree st;
// st.update(st.root, 1, N, pos, val);
// st.query(st.root, 1, N, l, r);

struct DinamicSegTree {
    struct Node {
        int val;
        Node* left, *right;
        Node() : val(0), left(nullptr), right(nullptr)
        {}
    };

    Node* root = new Node();

    void update(Node* no, int l, int r, int pos, int
        val) {
        if (l == r) {
            no->val = val;
            return;
        }
        int m = l + (r - l) / 2;
        // Cria filho só quando necessário (dinâmico)
        if (pos <= m) {
            if (!no->left) no->left = new Node();
            update(no->left, l, m, pos, val);
        } else {
            if (!no->right) no->right = new Node();
            update(no->right, m + 1, r, pos, val);
        }
        // Filho inexistente contribui com 0
        no->val = (no->left ? no->left->val : 0) + (no

```

```

        ->right ? no->right->val : 0);
    }

    int query(Node* no, int l, int r, int ql, int qr) {
        if (!no || l > qr || r < ql) return 0;
        if (l >= ql && r <= qr) return no->val;
        int m = l + (r - l) / 2;
        return query(no->left, l, m, ql, qr) + query(no
            ->right, m + 1, r, ql, qr);
    }
};

```

3.10 Segment Tree 2D dinâmica

```

// Segment Tree nas Linhas, outra dinâmica nas colunas
// O(log^2 N) por operação, O(Q log^2 N) espaço
//
// Quando usar:
// - Grid N x N com N grande (até 1e9) e poucas
// operações
// - Queries do tipo: soma em retângulo (x1,y1)-(x2,y2)
//
// Uso:
// SegTree2D st(N); // N = limite das coordenadas
// st.update(x, y, val);
// st.query(x1, y1, x2, y2);

struct SegTree2D {
    struct NodeY {
        int val;
        NodeY* left, *right;
        NodeY() : val(0), left(nullptr), right(nullptr)
        {}
    };

    struct NodeX {
        NodeY* yRoot;
        NodeX* left, *right;
        NodeX() : yRoot(new NodeY()), left(nullptr),
            right(nullptr) {}
    };

    NodeX* root;
    int N;

    SegTree2D(int N) : N(N), root(new NodeX()) {}

    // Atualiza coluna y com delta na árvore Y do nó X
    void updateY(NodeY* no, int l, int r, int y, int
        delta) {
        if (l == r) {
            no->val += delta;
            return;
        }
        int m = l + (r - l) / 2;
        if (y <= m) {
            if (!no->left) no->left = new NodeY();
            updateY(no->left, l, m, y, delta);
        } else {
            if (!no->right) no->right = new NodeY();
            updateY(no->right, m + 1, r, y, delta);
        }
        no->val = (no->left ? no->left->val : 0) + (no
            ->right ? no->right->val : 0);
    }

    int queryY(NodeY* no, int l, int r, int ql, int qr)
        {
        if (!no || l > qr || r < ql) return 0;
        if (l >= ql && r <= qr) return no->val;
        int m = l + (r - l) / 2;
        return queryY(no->left, l, m, ql, qr)
            + queryY(no->right, m + 1, r, ql, qr);
    }

    // Desce na árvore X atualizando a árvore Y de cada
    // nó afetado
    void updateX(NodeX* no, int l, int r, int x, int y,
        int delta) {
        updateY(no->yRoot, l, N, y, delta); // atualiza
        este nó X
    }
};

```

```

    if (l == r) return;
    int m = l + (r - 1) / 2;
    if (x <= m) {
        if (!no->left) no->left = new NodeX();
        updateX(no->left, l, m, x, y, delta);
    }
    else {
        if (!no->right) no->right = new NodeX();
        updateX(no->right, m + 1, r, x, y, delta);
    }
}

int queryX(NodeX* no, int l, int r, int x1, int xr,
            int y1, int yr) {
    if (!no || l > xr || r < x1) return 0;
    if (l >= x1 && r <= xr) return queryY(no->yRoot,
        1, N, y1, yr);
    int m = l + (r - 1) / 2;
    return queryX(no->left, l, m, x1, xr, y1, yr)
        + queryX(no->right, m + 1, r, x1, xr, y1,
            yr);
}

// Adiciona delta em (x, y)
void update(int x, int y, int delta) {
    updateX(root, 1, N, x, y, delta);
}

// Soma em retângulo (x1,y1)-(x2,y2)
int query(int x1, int y1, int x2, int y2) {
    return queryX(root, 1, N, x1, x2, y1, y2);
}
};

```

4 Grafos

4.1 Ordenação Topológica

```

// O(V + E)
vector<vector<int>> graph;
vector<int> visited, processing, order;
bool hasCycle = false;

void dfs(int u, int p) {
    visited[u] = true;
    processing[u] = true;

    for (auto& v : graph[u]) {
        if (v == p) continue; // necessario em grafo nao
            // direcionado;
        if (processing[v]) {
            hasCycle = true;
            return;
        }
        if (!visited[v])
            dfs(v);
    }

    processing[u] = false;
    order.push_back(u);
}

// Detecta e imprime qlqr ciclo encontrado
vector<int> cycle;
bool buscarCiclo(int u) {
    if (processing[u] && cycle.empty()) {
        cycle.push_back(u);
        return cycle.size() == 1;
    }
    if (visited[u]) return false;

    visited[u] = true;
    processing[u] = true;

    for (auto& v : graph[u]) {
        if (buscarCiclo(v)) {
            cycle.push_back(u);
            return cycle.back() != cycle.front();
        }
    }

    processing[u] = false;
}

```

```

    return false;
}

int main() {
    for (int i = 1; i <= n; ++i)
        if (dfs(i)) break; // para de procurar ciclos
            // se achar um

    if (cycle.size()) { // imprime ciclo achado
        cout << cycle.size() << '\n';
        reverse(cycle.begin(), cycle.end());

        for (auto& i : cycle)
            cout << i << ' ';
        cout << '\n';
    }

    return 0;
}

```

4.2 Dijkstra

```

// O((V + E)LogV) ou O(E LogV) - S(V)
const int N = 2e5 + 1, INF = 1e18;
vector<vector<pair<int, int>>> graph;
vector<int> dist;
int n, m;

void dijkstra(int s) {
    dist.assign(n + 1, INF);
    priority_queue<pair<int, int>> pq;
    dist[s] = 0;
    pq.push({0, s});

    while (!pq.empty()) {
        auto [du, u] = pq.top();
        pq.pop();
        if (-du > d[u]) continue; // otimização

        // v = vizinho, w = peso
        for (auto &[v, w] : graph[u]) {
            if (d[u] + w < d[v]) {
                d[v] = d[u] + w;
                pq.push({-d[v], v}); // -d[v] p manter
                    // ordenação crescente
            }
        }
    }
}

```

4.3 Bellman-Ford

```

// Calcula a menor distancia
// entre a e todos os vertices a
// detecta ciclo negativo
// Retorna 1 se ha ciclo negativo
// Nao precisa representar o grafo,
// soh armazenar as arestas
//
// Para achar ciclo positivo basta
// negar todos os pesos, logo
// um ciclo positivo vira negativo
//
// Para caminho máximo:
// Inicializar dist com -INF
// e funcao agora detecta ciclo
// positivo na n-ésima iteração
//
// O(V * E)

const int INF = 1e18; // 1e9 para int
struct Edge {
    int a, b, c;
};
vector<Edge> arestas;
vector<int> dist, pai; // pai[i] = no anterior a i
int n, m;

bool bellman_ford(int s) {
    dist.assign(n + 1, INF);
    pai.assign(n + 1, -1);
}

```



```

dist[s] = 0;

int ultimo_relaxado;
for (int i = 0; i < n; ++i) {
    ultimo_relaxado = -1;
    for (Edge& aresta : arestas) {
        if (dist[aresta.a] < INF && dist[aresta.b]
            > dist[aresta.a] + aresta.c) {
            dist[aresta.b] = dist[aresta.a] +
                aresta.c;
            pai[aresta.b] = aresta.a;
            ultimo_relaxado = aresta.b;
        }
    }
}

return ultimo_relaxado != -1; // true se tem ciclo
    negativo
}

// Retorna true se existe qualquer ciclo negativo
bool acharCicloNegativo() {
    dist.assign(n + 1, 0);
    pai.assign(n + 1, -1);

    int ciclo;
    for (int i = 0; i < n; ++i) {
        ciclo = -1;
        for (Edge& a : arestas) {
            if (dist[a.a] < INF && dist[a.b] > dist[a.a]
                + a.c) {
                dist[a.b] = dist[a.a] + a.c;
                pai[a.b] = a.a;
                ciclo = a.b;
            }
        }
    }

    if (ciclo == -1) return false; // não tem ciclo
        negativo

    for (int i = 0; i < n; ++i) ciclo = pai[ciclo];

    stack<int> caminho;
    for (int atual = ciclo; atual = pai[atual]) {
        caminho.push(atual);
        if (atual == ciclo && caminho.size() > 1)
            break;
    }
    while (!caminho.empty()) {
        cout << caminho.top() << ' ';
        caminho.pop();
    }
    cout << '\n';

    return true;
}

```

4.4 SPFA

```

// Shortest Path Faster Algorithm
// Bellman-Ford com fila – relaxa só vértices que
// mudaram
//  $O(V * E)$  pior caso,  $O(E)$  caso médio na prática
//
// Detecta ciclo negativo contando quantas vezes
// cada vértice entra na fila ( $> N$  vezes = ciclo)
//
// Representação: Lista de adjacência (diferente do
// Bellman-Ford)
// Mais eficiente que Bellman-Ford em grafos esparsos
//
// Para caminho máximo: negar todos os pesos
// Para ciclo positivo: negar todos os pesos e detectar
// ciclo negativo

const int INF = 1e18;
vector<pair<int, int>> adj[MxN]; // adj[u] = {v, peso}
vector<int> dist, pai;
vector<int> cnt; // quantas vezes cada vértice entrou
    na fila
vector<bool> inQueue;

```

```

int n, m;

// Retorna true se há ciclo negativo alcançável a
// partir de s
bool spfa(int s) {
    dist.assign(n + 1, INF);
    pai.assign(n + 1, -1);
    cnt.assign(n + 1, 0);
    inQueue.assign(n + 1, false);

    dist[s] = 0;
    queue<int> q;
    q.push(s);
    inQueue[s] = true;
    cnt[s] = 1;

    while (!q.empty()) {
        int u = q.front(); q.pop();
        inQueue[u] = false;

        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pai[v] = u;
                if (!inQueue[v]) {
                    q.push(v);
                    inQueue[v] = true;
                    cnt[v]++;
                    // Vértice entrou na fila mais de N
                    // vezes → ciclo negativo
                    if (cnt[v] > n) return true;
                }
            }
        }
    }

    return false;
}

// Retorna true se existe qualquer ciclo negativo no
// grafo
// (incluindo componentes não alcançáveis por s)
// Inicializa dist=0 para todos: simula supersource
bool acharCicloNegativo() {
    dist.assign(n + 1, 0);
    pai.assign(n + 1, -1);
    cnt.assign(n + 1, 0);
    inQueue.assign(n + 1, false);

    queue<int> q;
    for (int i = 1; i <= n; ++i) {
        q.push(i);
        inQueue[i] = true;
        cnt[i] = 1;
    }

    while (!q.empty()) {
        int u = q.front(); q.pop();
        inQueue[u] = false;

        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pai[v] = u;
                if (!inQueue[v]) {
                    q.push(v);
                    inQueue[v] = true;
                    cnt[v]++;
                    if (cnt[v] > n) return true;
                }
            }
        }
    }

    return false;
}

```

4.5 Floyd-Warshall

```

//  $O(V^3)$ ;  $S(V^2)$ 
int n, m;
void floyd_warshall(vector<vector<int>>& d) {
    for (int k = 1; k <= n; ++k)
        for (int i = 1; i <= n; ++i)

```

```

        for (int j = 1; j <= n; ++j)
            if (d[i][k] < INF && d[k][j] < INF)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
    }
// no main:
vector<vector<int>> d(n + 1, vector<int>(n + 1, INF));
for (int i = 1; i <= n; ++i)
    d[i][i] = 0; // dist = 0 para auto-laço

```

4.6 Caminho Euleriano

```

// Algoritmo de Hierholzer
// Para grafos não direcionados:
// O(V + E)

vector<vector<pair<int, int>>> adj; // {vizinho, id_aresta}
vector<bool> usado;
int n, m;

int temCaminhoEuleriano() {
    int ini = -1, impar = 0;

    for (int i = 1; i <= n; i++) {
        if (!adj[i].empty() && ini == -1) ini = i;
        if (adj[i].size() & 1)
            impar++, ini = i;
    }

    if (ini == -1) return 2;
    if (impar != 0 && impar != 2) return 0;

    vector<int> ptr(n + 1, 0); // próxima aresta a visitar
    stack<int> st;
    vector<int> caminho;
    st.push(ini);

    while (!st.empty()) {
        int u = st.top();
        // Avança ptr até achar aresta não usada
        while (ptr[u] < (int)adj[u].size() && usado[adj[u][ptr[u]].second])
            ptr[u]++;
        if (ptr[u] < (int)adj[u].size()) {
            auto [v, id] = adj[u][ptr[u]++];
            usado[id] = true;
            st.push(v);
        }
        else {
            caminho.push_back(u);
            st.pop();
        }
    }

    for (int i = 0; i < m; i++)
        if (!usado[i]) return 0;

    reverse(caminho.begin(), caminho.end());
    for (auto i : caminho) cout << i << ' ';
    cout << '\n';

    return impar == 0 ? 2 : 1;
}

// Para grafos direcionados:
// O(V + E)
vector<vector<int>> adj;
vector<int> in, out;
int n;
int temCaminhoEuleriano() {
    int ini = -1, fim = -1, temCiclo = 1;

    for (int i = 1; i <= n; i++) {
        if (out[i] - in[i] == 1) {
            if (ini != -1) return 0; // Invalido: soh 1 no pode ter out > in
            ini = i;
        }
        else if (in[i] - out[i] == 1) {
            if (fim != -1) return 0; // Invalido: soh 1

```

```

        no pode ter in > out
        fim = i;
    }
    else if (in[i] != out[i])
        return 0; // Invalido: |in-out| de um no NAO pode ser > 1
}

// Se não achou ini, é circuito
if (ini == -1) {
    temCiclo = 2;
    for (int i = 1; i <= n; i++) {
        if (out[i] > 0) {
            ini = i;
            break;
        }
    }
}

if (ini == -1) return 0;

// Algoritmo de Hierholzer
stack<int> st;
vector<int> caminho;
st.push(ini);

while (!st.empty()) {
    int u = st.top();
    if (!adj[u].empty()) {
        int v = adj[u].back();
        adj[u].pop_back();
        st.push(v);
    }
    else {
        caminho.push_back(u);
        st.pop();
    }
}

// Verifica se o grafo é conexo
for (int i = 1; i <= n; i++)
    if (!adj[i].empty()) return 0;

reverse(caminho.begin(), caminho.end());
for (auto& i : caminho)
    cout << i << ' ';
cout << '\n';

return temCiclo; // 1 = Caminho, 2 = Ciclo
}

```

4.7 Componentes Fortemente Conexos (SCC)

```

// O(V + E) - Algoritmo de Kosaraju
// Encontra todos os componentes fortemente conexos em grafo direcionado

const int MAXN = 1e5+1;
vector<vector<int>> grafo, grafoReverso;
vector<int> ordem; // Ordem topológica
bitset<MAXN> visitado;
int n, m; // vértices, arestas

// ordenação topológica
void dfsOrdem(int u) {
    visitado[u] = true;
    for (auto& v : grafo[u])
        if (!visitado[v])
            dfsOrdem(v);
    ordem.push_back(u);
}

// DFS no grafo reverso para encontrar SCCs
void dfsComponente(int u) {
    visitado[u] = true;
    for (auto& v : grafoReverso[u])
        if (!visitado[v])
            dfsComponente(v);
}

// Retorna vetor com um representante de cada componente

```



```
vector<int> kosaraju() {
    for (int i = 1; i <= n; ++i)
        if (!visitado[i])
            dfsOrdem(i);
    reverse(ordem.begin(), ordem.end());

    visitado.reset();
    vector<int> componentes;

    for (auto& u : ordem) {
        if (!visitado[u]) {
            dfsComponente(u);
            componentes.push_back(u);
        }
    }

    return componentes;
}
```

4.8 Árvore Geradora Mínima (MST)

```
// Prim
// O((V + E) * LogV)

const int MAXN = 1e5+1;
bitset<MAXN> visitado;

int prim() {
    priority_queue<pair<int, int>> pq; // {peso, no}
    pq.emplace(0, 1); // começa no no 1

    int sum = 0;
    while (!pq.empty()) {
        auto [w, u] = pq.top();
        pq.pop();

        if (visitado[u]) continue;

        sum += -w;
        visitado[u] = true;

        for (auto& [v, w] : adj[u]) // adj = {no, peso}
            if (!visitado[v])
                pq.emplace(-w, v);
    }

    return visitado[n] ? sum : -1;
}

// Kruskal (usa Union-find)
// O(E * LogE)
// N precisa guardar grafo, soh arestas
struct Edge {
    int a, b, c;
};
vector<Edge> arestas;
vector<int> p;

int kruskal() {
    for (int i = 1; i <= n; ++i) p[i] = i;
    sort(arestas.begin(), arestas.end(), [](auto a, auto b) {
        return a.c < b.c;
    });

    int sum = 0;
    for (auto& e : arestas) {
        if (find(e.a) != find(e.b)) {
            join(e.a, e.b);
            sum += e.c;
        }
    }

    return find(1) == find(n) ? sum : -1;
}

int find(int x) {
    if (x == p[x]) return x;
    return p[x] = find(p[x]);
}

void join(int x, int y) {
```

```
p[find(x)] = find(y);
}
```

5 Programação Dinâmica

5.1 Edit Distance

```
// Levenshtein Distance
// Problema: menor custo para transformar string s em t
// Operações: inserir (custo 1), remover (custo 1),
// substituir (custo (s[i]!=t[j]))
// DP 2D: O(N*M) tempo, O(N*M) espaço
int editDistance(string& s, string& t) {
    int n = s.size(), m = t.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 1e9));
    dp[0][0] = 0;
    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= m; j++) {
            if (i) dp[i][j] = min(dp[i][j], dp[i - 1][j] + 1); // remove
            if (j) dp[i][j] = min(dp[i][j], dp[i][j - 1] + 1); // insere
            if (i && j) dp[i][j] = min(dp[i][j], dp[i - 1][j - 1] + (s[i - 1] != t[j - 1])); // substitui
        }
    }
    return dp[n][m];
}

// DP 1D: O(N*M) tempo, O(M) espaço
int editDistance1D(string& s, string& t) {
    int n = s.size(), m = t.size();
    vector<int> prev(m + 1), cur(m + 1);
    iota(prev.begin(), prev.end(), 0);
    for (int i = 1; i <= n; i++) {
        cur[0] = i;
        for (int j = 1; j <= m; j++) {
            cur[j] = min({prev[j] + 1, cur[j - 1] + 1, prev[j - 1] + (s[i - 1] != t[j - 1])});
        }
        swap(prev, cur);
    }
    return prev[m];
}
```

5.2 Problema da Mochila

```
// Knapsack 0/1 - cada item usado no máximo 1 vez
// dp[j] = maior valor com capacidade exatamente j
// Itera j de trás pra frente: evita usar o item duas vezes
// O(N * W)
void knapsack01(int n, int W, vector<int>& w, vector<int>& v) {
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < n; i++)
        for (int j = W; j >= w[i]; j--)
            dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    cout << dp[W] << '\n';
}

// Knapsack ilimitado - cada item pode ser usado infinitas vezes
// Itera j da frente pra trás: permite reusar o mesmo item
// O(N * W)
void knapsack(int n, int W, vector<int>& w, vector<int>& v) {
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < n; i++)
        for (int j = w[i]; j <= W; j++)
            dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    cout << dp[W] << '\n';
}

// Knapsack 0/1 - quando W é muito grande, inverte dimensões
// dp[j] = menor peso para atingir exatamente valor j
// Útil quando V (soma dos valores) << W
// O(N * V)
```

```
void knapsack01ByValue(int n, int W, vector<int>& w,
    vector<int>& v) {
    int total = 0;
    for (int x : v) total += x;
    vector<int> dp(total + 1, INF / 2);
    dp[0] = 0;
    for (int i = 0; i < n; i++)
        for (int j = total; j >= v[i]; j--)
            dp[j] = min(dp[j], dp[j - v[i]] + w[i]);
    int ans = 0;
    for (int j = 0; j <= total; j++)
        if (dp[j] <= W) ans = j;
    cout << ans << '\n';
}
```

5.3 Longest Increasing Subsequence (LIS)

```
// O(N * LogN)
int lis(vector<int>& v) {
    int n = v.size();
    vector<int> ans;

    // Initialize the answer vector with the
    // first element of v
    ans.push_back(v[0]);

    for (int i = 1; i < n; i++) {
        if (v[i] > ans.back()) {
            ans.push_back(v[i]);
        }
        else {
            int low = lower_bound(ans.begin(), ans.end(),
                v[i]) - ans.begin();
            ans[low] = v[i];
        }
    }

    return ans.size();
}
```

5.4 Máscara de Bits

```
// Operações com Bitmask
// Índices 0-based. "bit i" = posição i da direita.
// Uso típico: representar subconjuntos de {0, 1, ..., N-1}

// --- Operações básicas ---
mask | (1 << i) // liga bit i
mask & ~(1 << i) // desliga bit i
mask ^ (1 << i) // inverte bit i
(mask >> i) & 1 // lê bit i (0 ou 1)
mask & (1 << i) // testa bit i (0 se desligado)

bool isPowerOf2(int &n) {
    return (n & (n - 1)) == 0;
}

int contarBits(int n) {
    int ans = 0;
    while (n > 0) {
        n = n & (n - 1);
        ans++;
    }
    return ans;
}

int to_binary(int n) {
    int ans = 0;
    int p = 1;
    while (n > 0) {
        int last_bit = (n & 1);
        ans += p * last_bit;
        p *= 10;
        n >>= 1;
    }
    return ans;
}

void limparUltimosIBits(int &n, int i) {
    int mask = (-1 << i);
    n = n & mask;
}

void limparRangeBits(int &n, int i, int j) {
```

```
int a = (~0) << (j + 1); // ~0 == -1
int b = (1 << i) - 1;
int mask = a | b;
n = n & mask;
}

// --- Propriedades do conjunto ---
__builtin_popcount(mask) // quantos bits ligados
__builtin_ctz(mask) // índice do bit menos
significativo ligado
__builtin_clz(mask) // zeros à esquerda (cuidado:
// UB se mask==0)
(mask & (mask - 1)) == 0 // mask é potência de 2 (
// exatamente 1 bit)
mask == 0 // conjunto vazio

// --- Iteração ---

// Todos os subconjuntos de {0..N-1}
for (int mask = 0; mask < (1 << n); mask++) { }

// Todos os bits ligados de mask
for (int tmp = mask; tmp; tmp &= tmp - 1) {
    int i = __builtin_ctz(tmp); // índice do bit
}

// Todos os subconjuntos de mask (incluindo vazio)
for (int sub = mask; sub > 0; sub = (sub - 1) & mask) { }

// Atenção: não itera o subconjunto vazio; adicione
// caso sub==0 se necessário

// --- Máscaras úteis ---
(1 << n) - 1 // todos os n bits ligados
(1 << i) // só o bit i ligado

// --- Padrão DP em subconjuntos ---
// dp[mask][v] = valor considerando exatamente os
// vértices em mask,
// terminando/passando em v
// Transição típica:
// int prev = mask ^ (1 << v); // remove v do
// conjunto
// dp[mask][v] = f(dp[prev][u]) para u vizinho de v
// em prev

// Exemplo (Hamiltonian Path - O(2^N * N^2)):
// dp[mask][v] += dp[mask ^ (1 << v)][u] * adj[u][v]
// para todo u em (mask ^ (1 << v)) com adj[u][v] > 0
```

6 Matemática

6.1 Fórmulas

PROGRESSÕES E SOMAS

Soma de consecutivos (PA):

$$S = (\text{primeiro} + \text{último}) * n / 2$$

Soma de 1 até N:

$$S = N * (N + 1) / 2$$

Soma de quadrados de 1 até N:

$$S = N * (N + 1) * (2N + 1) / 6$$

Soma de cubos de 1 até N:

$$S = (N * (N + 1) / 2)^2$$

Soma de PG:

$$S = a * (r^n - 1) / (r - 1) \quad (r \neq 1)$$

Soma de PG infinita ($|r| < 1$):

$$S = a / (1 - r)$$

Termo geral da PA:

$$a_n = a_1 + (n - 1) * d$$

Termo geral da PG:

$$a_n = a_1 * r^{(n-1)}$$

COMBINATÓRIA

Fatorial:

$$n! = n * (n-1) * \dots * 1$$

$$0! = 1$$

Combinação:

$$C(n, k) = n! / (k! * (n-k)!)$$

$$C(n, 0) = C(n, n) = 1$$

$$C(n, k) = C(n-1, k-1) + C(n-1, k) \quad (\text{triângulo de Pascal})$$

Permutação:

$$P(n, k) = n! / (n-k)!$$

Permutação com repetição:

$$P = n! / (n_1! * n_2! * \dots * n_k!)$$

Arranjo:

$$A(n, k) = n! / (n-k)!$$

Número de subconjuntos de N elementos:

$$2^N$$

Princípio da inclusão-exclusão:

$$|A \cup B| = |A| + |B| - |A \cap B|$$

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$$

Números de Catalan:

$$C_n = C(2n, n) / (n + 1)$$

$$C_0=1, C_1=1, C_2=2, C_3=5, C_4=14, \dots$$

Uso: árvores binárias, parênteses válidos, triangulações

Coeficiente binomial – identidades úteis:

$$C(n, k) = C(n, n-k)$$

$$C(n, k) \text{ para } k=0..n = 2^n$$

$$C(n, k)^2 \text{ para } k=0..n = C(2n, n)$$

Números de Stirling (segunda espécie):

$$S(n, k) = \text{número de formas de particionar } n \text{ elementos em } k \text{ grupos não vazios}$$

$$S(n, k) = k * S(n-1, k) + S(n-1, k-1)$$

Números de Bell:

$$B_n = \text{número total de partições de um conjunto de } n \text{ elementos}$$

$$B_n = \sum S(n, k) \text{ para } k=0..n$$

Derangements (permutações sem ponto fixo):

$$D(n) = (n-1) * (D(n-1) + D(n-2))$$

$$D(n) = n! * \sum (-1)^k / k! \text{ para } k=0..n$$

$$D(0)=1, D(1)=0, D(2)=1, D(3)=2, D(4)=9$$

Lema de Burnside:

$$|X/G| = (1/|G|) * \sum |\text{Fix}(g)| \text{ para } g \text{ em } G$$

TEORIA DOS NÚMEROS

Divisibilidade – número de divisores:

$$\text{Se } n = p_1^{a_1} * p_2^{a_2} * \dots * p_k^{a_k}$$

$$d(n) = (a_1+1) * (a_2+1) * \dots * (a_k+1)$$

Soma dos divisores:

$$\sigma(n) = \prod (p_i^{a_i+1} - 1) / (p_i - 1)$$

Função de Euler (totiente):

$$\phi(n) = n * \prod (1 - 1/p) \text{ para } p \text{ primo divisor de } n$$

$$\phi(1) = 1$$

$$\text{Se } p \text{ primo: } \phi(p) = p - 1$$

$$\text{Se } p \text{ primo: } \phi(p^k) = p^k - p^{k-1}$$

$$(d) \text{ para } d \text{ divisor de } n = n$$

Pequeno Teorema de Fermat:

$$a^p \equiv a \pmod{p} \text{ para } p \text{ primo}$$

$$a^{p-1} \equiv 1 \pmod{p} \text{ para } p \text{ primo e } \text{mdc}(a, p)=1$$

Teorema de Euler:

$$a^{\phi(n)} \equiv 1 \pmod{n} \text{ para } \text{mdc}(a, n)=1$$

Inverso modular (p primo):

$$a^{p-1} \equiv 1 \pmod{p}$$

Teorema Chinês do Resto (CRT):

Sistema $x \equiv a_i \pmod{m_i}$ com m_i coprimos dois a dois:

$$x = \sum a_i * M_i * y_i \pmod{M}$$

onde $M = \prod m_i$, $M_i = M/m_i$, $y_i = M_i^{-1} \pmod{m_i}$

MDC / MMC:

$$\text{mdc}(a, b) = \text{mdc}(b, a \bmod b)$$

$$\text{mmc}(a, b) = a * b / \text{mdc}(a, b)$$

$$\text{mdc}(a, 0) = a$$

Identidade de Bézout:

$$\text{mdc}(a, b) = a*x + b*y \quad (x, y \text{ inteiros})$$

Teorema de Wilson:

$$(p-1)! \equiv -1 \pmod{p} \text{ se e somente se } p \text{ é primo}$$

Teorema de Wolstenholme:

$$\text{Para } p \text{ primo } \geq 5, C(2p, p) \equiv 2 \pmod{p^3}$$

Contagem de primos até N (aproximação, Teorema dos

Números Primos):

$$\pi(N) \approx N / \ln(N)$$

Fórmula de Legendre (expoente de p em n!):

$$e_p(n!) = \sum \text{floor}(n / p^k) \text{ para } k=1,2,3,\dots$$

GEOMETRIA

Distância euclidiana:

$$d = \sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$$

Distância Manhattan:

$$d = |x_2-x_1| + |y_2-y_1|$$

Distância Chebyshev:

$$d = \max(|x_2-x_1|, |y_2-y_1|)$$

Área do triângulo (coordenadas):

$$A = |x_1(y_2-y_3) + x_2(y_3-y_1) + x_3(y_1-y_2)| / 2$$

Produto vetorial 2D:

$$(a \times b) = a.x * b.y - a.y * b.x$$

> 0: b está à esquerda de a (anti-horário)

< 0: b está à direita de a (horário)

= 0: colineares

Produto escalar:

$$a \cdot b = a.x * b.x + a.y * b.y = |a| |b| \cos(\theta)$$

Área do polígono (Shoelace):

$$A = \sum | (x_i * y_{i+1}) - (x_{i+1} * y_i) | / 2$$

Teorema de Pick (polígono em grade inteira):

$$A = I + B/2 - 1$$

I = pontos inteiros internos

B = pontos inteiros na borda

A = área (Shoelace)

--- Triângulos ---

Área (base x altura):

$$A = \text{base} * \text{altura} / 2$$

Fórmula de Herão (lados a,b,c):

$$s = (a + b + c) / 2$$

$$A = \sqrt{s * (s-a) * (s-b) * (s-c)}$$

Área via lei dos senos:

$$A = (a * b * \sin(C)) / 2$$

Lei dos cossenos:

$$c^2 = a^2 + b^2 - 2*a*b*\cos(C)$$

Lei dos senos:

$$a/\sin(A) = b/\sin(B) = c/\sin(C) = 2R$$

(R = raio **do** circuncírculo)

Triângulo isósceles (lados iguais L, base B):

$$\text{altura } h = \sqrt{L^2 - (B/2)^2}$$

$$A = B * h / 2$$

Triângulo equilátero (lado L):

$$A = (\sqrt{3}/4) * L^2$$

$$\text{altura } h = (\sqrt{3}/2) * L$$

Raio **do** incírculo:

$$r = A / s \quad (s = \text{semiperímetro})$$

Raio **do** circuncírculo:

$$R = (a * b * c) / (4 * A)$$

Centroide:

$$G = ((x_1+x_2+x_3)/3, (y_1+y_2+y_3)/3)$$

Soma dos ângulos internos de um triângulo:

$$180^\circ$$

--- Polígonos ---

Soma dos ângulos internos de um polígono de N lados:

$$S = (N - 2) * 180^\circ$$

Ângulo interno de polígono regular de N lados:

$$a = (N - 2) * 180^\circ / N$$

Ângulo externo de polígono regular:

$$360^\circ / N$$

Área de polígono regular (lado L, N lados):

$$A = (N * L^2) / (4 * \tan(\pi/N))$$

Apótema de polígono regular:

$$ap = L / (2 * \tan(\pi/N))$$

Área de polígono regular (via apótema e perímetro):

$$A = ap * \text{perímetro} / 2$$

Diagonais de um polígono de N lados:

$$D = N * (N - 3) / 2$$

--- Quadriláteros ---

Retângulo:

$$A = \text{base} * \text{altura}$$

$$\text{Diagonal} = \sqrt{\text{base}^2 + \text{altura}^2}$$

Quadrado (lado L):

$$A = L^2$$

$$\text{Diagonal} = L * \sqrt{2}$$

Losango (diagonais d1, d2):

$$A = d_1 * d_2 / 2$$

Trapézio (bases b1, b2, altura h):

$$A = (b_1 + b_2) * h / 2$$

Fórmula de Brahmagupta (quadrilátero cíclico, lados a, b, c, d):

$$s = (a+b+c+d)/2$$

$$A = \sqrt{(s-a)*(s-b)*(s-c)*(s-d)}$$

--- Círculo ---

Área:

$$A = \pi * r^2$$

Perímetro (circunferência):

$$C = 2 * \pi * r$$

Setor circular (ângulo θ em radianos):

$$\text{Área} = \theta * r^2 / 2$$

$$\text{Comprimento } \text{do} \text{ arco} = \theta * r$$

Segmento circular (ângulo θ em radianos):

$$\text{Área} = r^2 * \theta(1 - \sin(\theta/2)) / 2$$

Comprimento de corda (ângulo central θ):

$$L = 2 * r * \sin(\theta/2)$$

Área entre duas circunferências concêntricas (coroa circular):

$$A = \pi * (R^2 - r^2)$$

Potência de um ponto em relação a um círculo:

$$\text{pot} = d^2 - r^2 \quad (d = \text{distância } \text{do} \text{ ponto ao centro})$$

--- Sólidos (área e volume) ---

Cubo (aresta a):

$$V = a^3$$

$$A_{\text{total}} = 6*a^2$$

Paralelepípedo (a,b,c):

$$V = a*b*c$$

$$A_{\text{total}} = 2*(ab + bc + ac)$$

Esfera (raio r):

$$V = (4/3) * \pi * r^3$$

$$A_{\text{total}} = 4 * \pi * r^2$$

Cilindro (raio r, altura h):

$$V = \pi * r^2 * h$$

$$A_{\text{lateral}} = 2 * \pi * r * h$$

$$A_{\text{total}} = \pi * 2 * r * h + \pi * 2 * r^2$$

Cone (raio r, altura h, geratriz g = $\sqrt{r^2+h^2}$):

$$V = \pi * r^2 * h / 3$$

$$A_{\text{lateral}} = \pi * r * g$$

$$A_{\text{total}} = \pi * r * g + \pi * r^2$$

Pirâmide (área da base Ab, altura h):

$$V = Ab * h / 3$$

Prisma (área da base Ab, altura h):

$$V = Ab * h$$

Tronco de cone (raios R, r, altura h):

$$V = \pi * h / 3 * (R^2 + R*r + r^2)$$

--- Geometria analítica / vetores ---

Ponto médio:

$$M = ((x_1+x_2)/2, (y_1+y_2)/2)$$

Equação da reta (dois pontos):

$$(y - y_1) / (y_2 - y_1) = (x - x_1) / (x_2 - x_1)$$

Distância de ponto a reta ($ax+by+c=0$):

$$d = |a*x_0 + b*y_0 + c| / \sqrt{a^2 + b^2}$$

Interseção de segmentos:

usar orientação (produto vetorial) para checar se os segmentos se cruzam

Norma de um vetor:

$$|v| = \sqrt{v.x^2 + v.y^2}$$

Ângulo entre dois vetores:

$$\cos(\theta) = (a \cdot b) / (|a| * |b|)$$

Rotação de ponto θ (radianos, em torno da origem):

$$x' = x * \cos(\theta) - y * \sin(\theta)$$

$$y' = x * \sin(\theta) + y * \cos(\theta)$$

Fecho convexo (Convex Hull) – Teorema:

Complexidade ótima $O(n \log n)$ (ex: Graham Scan, Andrew's *Monotone Chain*)

TEOREMAS ÚTEIS DIVERSOS

Teorema de Pitágoras:

$$a^2 + b^2 = c^2 \quad (c = \text{hipotenusa})$$

Teorema de Tales:

retas paralelas cortadas por transversais formam segmentos proporcionais

Teorema da Bissetriz Interna:

divide o lado oposto em segmentos proporcionais aos lados adjacentes

Teorema de Ceva:

$(AF/FB) * (BD/DC) * (CE/EA) = 1$ (cevianas concorrentes)

Teorema de Menelaus:

$(AF/FB) * (BD/DC) * (CE/EA) = -1$ (pontos colineares em transversal)

Teorema de Stewart (triângulo com ceviana d até lado a , dividindo-o em m e n):

$b^2 * m + c^2 * n - d^2 * a = a * m * n$

TEORIA DOS GRAFOS

Handshaking lemma:

$\text{grau}(v) = 2 * |E|$

Árvore com N vértices:

$|E| = N - 1$

Grafo euleriano (não direcionado):

Circuito: todos os graus pares + conexo

Caminho: exatamente 2 vértices de grau ímpar + conexo

Grafo euleriano (direcionado):

Circuito: $\text{in}[v] = \text{out}[v]$ para todo v + conexo

Caminho: um v com $\text{out}-\text{in}=1$ (início), um com $\text{in}-\text{out}=1$ (fim), resto iguais

Fórmula de Euler (grafo planar):

$V - E + F = 2$ (V vértices, E arestas, F faces incluindo externa)

Número máximo de arestas em grafo planar simples ($N \geq 3$):

$E \leq 3N - 6$

Teorema de Kirchhoff (número de árvores geradoras):

via matriz Laplaciana, teorema da árvore geradora (Matrix-Tree Theorem)

PROBABILIDADE

Probabilidade condicional:

$P(A|B) = P(A \cap B) / P(B)$

Teorema de Bayes:

$P(A|B) = P(B|A) * P(A) / P(B)$

Valor esperado:

$E[X] = \sum x_i * P(x_i)$

$E[aX + b] = a * E[X] + b$

$E[X + Y] = E[X] + E[Y]$

Variância:

$\text{Var}(X) = E[X^2] - E[X]^2$

Linearidade da esperança:

vale mesmo se X e Y não forem independentes

EXPONENCIAÇÃO E LOGARITMOS

Exponenciação rápida:

a^n em $O(\log n)$ por repeated squaring

Identidades de log:

$\log(a * b) = \log(a) + \log(b)$

$\log(a / b) = \log(a) - \log(b)$

$\log(a^b) = b * \log(a)$

$\log_b(a) = \log(a) / \log(b)$

Número de dígitos de N na base B :

$d = \text{floor}(\log_B(N)) + 1$

BITMASK

Número de subconjuntos de N elementos:

2^N

Soma de todos os tamanhos de subconjuntos:

$N * 2^{(N-1)}$

STRINGS

Número de substrings de string de tamanho N :

$N * (N + 1) / 2$

MISCELÂNEA

Princípio das casas de pombo (Pigeonhole):

$N+1$ objetos em N caixas $\rightarrow$ alguma caixa tem ≥ 2 objetos

Soma dos primeiros N ímpares:

$1 + 3 + 5 + \dots + (2N-1) = N^2$

Potências de 2:

$2^{10} = 1.024$ ($\sim 10^3$)

$2^{20} = 1.048.576$ ($\sim 10^6$)

$2^{30} = 1.073.741.824$ ($\sim 10^9$)

Limite int (32 bits):

$\text{INT_MAX} = 2^{31} - 1 = 2.147.483.647$ ($\sim 2 * 10^9$)

$\text{INT_MIN} = -2^{31} = -2.147.483.648$

Limite long long (64 bits):

$\text{LLONG_MAX} = 2^{63} - 1$ ($\sim 9.2 * 10^{18}$)

$\text{LLONG_MIN} = -2^{63}$ ($\sim -9.2 * 10^{18}$)

6.2 MDC & MMC

// $O(\log N)$; $n = \min(a, b)$

// existe a função gcd()

int mdc(**int** a, **int** b) {

if (b == 0) **return** a;

return mdc(b, a % b);

}

// existe a função lcm()

int mmc(**int** a, **int** b) {

return (a * b) / mdc(a, b);

}

6.3 Exponenciação Binária

// a^b $O(\log N)$

int bin_pow(**int** a, **int** b, **const int** MOD = 1e9 + 7) {

a %= MOD;

int res = 1;

while (b) {

if (b % 2) res = res * a % MOD;

a = a * a % MOD;

b /= 2;

}

return res;

}

int inverso(**int** a, **int** MOD) {

bin_pow(a, MOD - 2); *// MOD deve ser primo*

}

6.4 Numeros de Armstrong

```
// Numero n de D digitos onde soma dos digitos^D == n
vector<long long> armstrong; // fica ordenado

// O(C(D+9,9))
void generate(int MAXD = 18) {
    for (int D = 1; D <= MAXD; D++) {
        long long pw[10];
        for (int d = 0; d <= 9; d++) {
            pw[d] = 1;
            for (int i = 0; i < D; i++) pw[d] *= d;
        }

        vector<int> cnt(10, 0);
        auto rec = [&](int idx, int rem) {
            if (idx == 9) {
                cnt[9] = rem;
                long long sum = 0;
                for (int d = 0; d <= 9; d++) sum += (
                    long long)cnt[d] * pw[d];

                string s = to_string(sum);
                if ((int)s.size() != D) return;

                int check[10] = {0};
                for (char c : s) check[c - '0']++;
                for (int d = 0; d <= 9; d++)
                    if (check[d] != cnt[d]) return;

                armstrong.push_back(sum);
                return;
            }
            for (int take = 0; take <= rem; take++) {
                cnt[idx] = take;
                rec(idx + 1, rem - take);
            }
        };
        rec(0, D);
    }
    sort(armstrong.begin(), armstrong.end());
}

// O(1)
bool is_armstrong(long long n) {
    return binary_search(armstrong.begin(), armstrong.end(), n);
}

// no main:
// generate(); // chamar 1 vez, cobre ate 18 digitos (
// long long)
```

6.5 Primos

mashu lucky prime : 91145149

Primos $\sim 10^8 / 10^9$ (hashing/mod):
 1097774749, 1076767633, 100102021, 999997771
 1001010013, 1000512343, 987654361, 999991231
 999888733, 98789101, 98777733, 999991921, 1010101333

MOD classicos:
 1000000007
 1000000009
 998244353

Primos pequenos (double hashing / tabelas):
 31, 37, 53, 61, 97, 131, 193, 257, 389, 769,
 1543, 3079, 6151, 12289, 24593, 49157, 98317,
 196613, 393241, 786433

6.6 Crivo de Eratóstenes

```
// O(N * Log(LogN))
vector<bool> eh_primo(TAM, true);
void crivo(int n) {
    eh_primo[0] = eh_primo[1] = false;
    for (int p = 2; p * p <= n; p++) {
        if (eh_primo[p]) {
            for (int i = p * p; i <= n; i += p)
                eh_primo[i] = false;
        }
    }
}
```

6.7 Fatoração Prima

```
// --- Fatoração de um único número ---
// Retorna mapa {primo -> expoente}
// Exemplo: 12 = 2^2 * 3^1 -> {{2,2},{3,1}}
// O(sqrt(N))
map<int,int> factor(int n) {
    map<int,int> fat;
    for (int p = 2; p * p <= n; p++) {
        while (n % p == 0) {
            fat[p]++;
            n /= p;
        }
    }
    // Fator primo restante > sqrt(N)
    if (n > 1) fat[n]++;
    return fat;
}

// --- Menor fator primo (SPF) ---
// Pré-computa spf[i] = menor fator primo de i
// Pré-processamento: O(N * Log(LogN))
// Fatoração de qualquer n <= N: O(LogN)
vector<int> spf;
void buildSPF(int n) {
    iota(spf.begin(), spf.end(), 0);
    for (int p = 2; p * p <= n; p++) {
        if (spf[p] == p) { // p é primo
            for (int j = p * p; j <= n; j += p)
                if (spf[j] == j)
                    spf[j] = p; // marca menor fator
        }
    }
}

map<int,int> factorSPF(int n) {
    map<int,int> fat;
    while (n > 1) {
        fat[spf[n]]++;
        n /= spf[n];
    }
    return fat;
}
```

6.8 Função Totiente de Euler (ϕ)

```
//  $\phi(n)$  = quantidade de inteiros em  $[1, n]$  coprimos com n
// Propriedades:
//  $\phi(1) = 1$ 
// p primo  $\rightarrow \phi(p) = p - 1$ 
// p primo  $\rightarrow \phi(p^k) = p^k - p^{(k-1)}$ 
//  $\text{mdc}(a,b)=1 \rightarrow \phi(a*b) = \phi(a) * \phi(b)$  (multiplicativa)
//
// Fórmula:  $\phi(n) = n * \prod (1 - 1/p)$  para todo primo  $p \mid n$ 
//  $= n * \prod (p-1)/p$ 
// Exemplo:  $\phi(12) = 12 * (1-1/2) * (1-1/3) = 12 * 1/2 * 2/3 = 4$ 
//
// Uso com exponenciação modular:
//  $a^b \text{ mod } c = a^{(b \% \phi(c) + \phi(c))} \text{ mod } c$  se  $b \geq \phi(c)$ 
//  $= a^b \text{ mod } c$  se  $b < \phi(c)$ 
//
// --- Totiente de um único número ---
// O(sqrt(N))
int phi(int n) {
    int res = n;
    // Fatoramos n e para cada primo p encontrado
    // aplicamos res = res * (p-1)/p
    // (equivalente a res -= res/p, evitando divisão com perda)
    for (int p = 2; p * p <= n; p++) {
        if (n % p == 0) {
            // p é fator primo de n
            res = res / p * (p - 1); // res *= (1 - 1/p)
            while (n % p == 0) n /= p; // remove todas as
            // ocorrências de p
        }
    }
```



```

}
// Se ainda sobrou um fator > 1, ele é primo
if (n > 1) res = res / n * (n - 1);
return res;
}

// --- Tabela de totientes para [1, n] ---
// O(N * log(logN)) - igual ao crivo de Eratóstenes
// Ideia: para cada primo p, multiplica todos os
// múltiplos por (p-1)/p
int phi_table[MAXN];
void phitable(int n) {
    // Inicializa φ(i) = i para todo i (valor antes de
    // aplicar os fatores)
    for (int i = 1; i <= n; i++) phi_table[i] = i;

    for (int p = 2; p <= n; p++) {
        // Se φ(p) ainda é p, então p é primo (não foi
        // modificado por nenhum fator)
        if (phi_table[p] == p) {
            // Aplica o fator (p-1)/p em todos os múltiplos de
            // p
            for (int j = p; j <= n; j += p)
                phi_table[j] = phi_table[j] / p * (p - 1);
        }
    }
}

```

6.9 Problema de Josefo (Josephus)

```

// === Josephus Problem ===
// n pessoas numeradas 0..n-1 em círculo, remove k-
// ésima a cada passo
// Retorna posição do sobrevivente (0-indexed)
//
// Complexidade: O(n) ingênuo, O(log n) para k=2, O(k
// log n) caso geral
//
// Fórmula clássica: josephus(n,k) = (josephus(n-1,k) +
// k) % n

// --- k fixo = 2 (recorrência eficiente O(log n)) ---
// Base: josephus(1) = 0
// josephus(2n) = 2*josephus(n) + 1
// josephus(2n+1) = 2*josephus(n) - 1
int josephus2(int n) {
    if (n == 1) return 0;
    if (n & 1) return 2 * josephus2(n / 2) + 1;
    return 2 * josephus2(n / 2) - 1;
}

// --- k arbitrário O(k log n) ---
int josephus(int n, int k) {
    if (n == 1) return 0;
    if (k == 1) return n - 1;
    if (k > n) return (josephus(n - 1, k) + k) % n;
    int cnt = n / k;
    int res = josephus(n - cnt, k);
    res -= n % k;
    if (res < 0) res += n;
    else res += res / (k - 1);
    return res;
}

// --- Versão iterativa O(n) para n pequeno ---
int josephus_iter(int n, int k) {
    int res = 0;
    for (int i = 2; i <= n; i++)
        res = (res + k) % i;
    return res; // 0-indexed
}

```

6.10 Teorema Chinês do Resto (CRT)

```

// Resolve sistema: x ≡ a[i] (mod m[i])
// Complexidade: O(N log M) onde M = prod(m[i])
// Retorna {x, M} ou {-1, -1} se sem solução

pair<int, int> crt(vector<int> &a, vector<int> &m) {
    int x = 0, M = 1;
    for (int i = 0; i < a.size(); i++) {

```

```

        int ai = a[i] % m[i];
        if (ai < 0) ai += m[i];

        // Resolve: M * t ≡ ai - x (mod m[i])
        int g = gcd(M, m[i]);
        if ((ai - x) % g != 0) return {-1, -1};

        int M_ = M / g, m_ = m[i] / g;
        int t = ((ai - x) / g) * inv_mod(M_ % m_, m_) % m_;
        if (t < 0) t += m_;

        x += t * M;
        M *= m[i] / g;
        x %= M;
    }
    return {x, M};
}

// Garner's algorithm: recupera x mod MOD dado x ≡ a[i]
// (mod m[i])
// Requer m[i] coprimos entre si
// Complexidade: O(N²)
int garner(vector<int> &a, vector<int> &m, int MOD) {
    int n = a.size();
    vector<int> x(n);
    for (int i = 0; i < n; i++) {
        x[i] = a[i];
        for (int j = 0; j < i; j++) {
            x[i] = (x[i] - x[j]) * inv_mod(m[j], m[i]) % m[i];
            if (x[i] < 0) x[i] += m[i];
        }
    }
    int ans = 0, mult = 1;
    for (int i = 0; i < n; i++) {
        ans = (ans + x[i] * mult) % MOD;
        mult = mult * m[i] % MOD;
    }
    return ans;
}

```

7 Strings

7.1 Suffix Array

```

// sa[i] = índice do i-ésimo menor sufixo de s (0-based)
// rank[i] = posição do sufixo i no SA
// lcp[i] = longest common prefix entre sa[i] e sa[i-1]
//
// Usos:
// - Busca de padrão: binary search no SA
// - Substring mais longa que aparece ≥ k vezes:
//   maior intervalo de LCP ≥ k-1
// - Número de substrings distintas: n*(n+1)/2 - sum(lcp)
// - Comparar sufixos rapidamente: rank[i] < rank[j]
//   => sufixo i < sufixo j
// - Maior substring comum entre strings: concatenar
//   com '#' e checar LCP
//
// O(N log N)
vector<int> build_sa(string s) {
    s += "$";
    int n = s.size();
    vector<int> sa(n), r(n), nr(n);
    iota(sa.begin(), sa.end(), 0);

    // ordena pelos primeiros caracteres
    sort(sa.begin(), sa.end(), [&](int i, int j) {
        return s[i] < s[j];
    });

    r[sa[0]] = 0;
    for (int i = 1; i < n; i++)
        r[sa[i]] = r[sa[i - 1]] + (s[sa[i]] != s[sa[i - 1]]);

    // dobra o tamanho do prefixo comparado
    for (int k = 1; k < n; k <= 1) {
        sort(sa.begin(), sa.end(), [&](int i, int j) {
            if (r[i] != r[j]) return r[i] < r[j];

```

```

    int ri = (i + k < n) ? r[i + k] : -1;
    int rj = (j + k < n) ? r[j + k] : -1;
    return ri < rj;
});

nr[sa[0]] = 0;
for (int i = 1; i < n; i++)
    nr[sa[i]] = nr[sa[i - 1]] + ((r[sa[i]] != r[sa[i - 1]]) ? 1 : 0);

// Kasai's algorithm O(N)
vector<int> build_lcp(string &s, vector<int> &sa) {
    int n = s.size();
    vector<int> rank(n), lcp(n);
    for (int i = 0; i < n; i++) rank[sa[i]] = i;

    int k = 0;
    for (int i = 0; i < n; i++) {
        if (rank[i] == n - 1) {
            k = 0; // último no SA (o '$')
            continue;
        }
        int j = sa[rank[i] + 1];
        while (i + k < n && j + k < n && s[i + k] == s[j + k]) k++;
        lcp[rank[i]] = k;
        if (k) k--;
    }
    return lcp; // lcp[i] = LCP entre sa[i] e sa[i-1], lcp[0] = 0
}

// Busca padrão p: retorna {primeira, última} posição no SA
// s deve ter '$' no final
// Quantidade de p no texto será second - first + 1, se existir p (first != -1)
// O(|p| * logN)
pair<int, int> find_pattern(string &s, vector<int> &sa, string &p) {
    int n = s.size(), m = p.size();
    int L = 0, R = n - 1;

    // Lower_bound
    int first = -1;
    while (L <= R) {
        int mid = (L + R) / 2;
        int cmp = s.compare(sa[mid], m, p);
        if (cmp >= 0) {
            if (cmp == 0) first = mid;
            R = mid - 1;
        } else L = mid + 1;
    }
    if (first == -1) return {-1, -1};

    // upper_bound
    L = 0, R = n - 1;
    int last = first;
    while (L <= R) {
        int mid = (L + R) / 2;
        int cmp = s.compare(sa[mid], m, p);
        if (cmp <= 0) {
            if (cmp == 0) last = mid;
            L = mid + 1;
        }
    }
    return {first, last};
}

```

```

    }
    else R = mid - 1;
}
return {first, last};
}

```

7.2 KMP

```

// p[i] = comprimento do maior prefixo próprio do
// padrão s que também é sufixo
// do texto t[0..i] Prefixo/Sufixo próprio é um prefixo
// /sufixo que não é a
// string inteira Usos:
// - Busca de padrão único (igual Z, mas mais natural
// para busca online)
// - Busca online/streaming: processa t caractere a
// caractere sem ter t
// completo
// - Menor período de string: n - lps[n-1] é o
// período mínimo
// - Autômato de string: transformar lps em DFA para
// múltiplas consultas no
// mesmo padrão

// O(m), m = s.size()
vector<int> pi(string s) {
    vector<int> p(s.size());
    for (int i = 1, j = 0; i < s.size(); ++i) {
        while (j > 0 && s[j] != s[i]) j = p[j - 1];
        if (s[j] == s[i]) j++;
        p[i] = j;
    }
    return p;
}

// O(n + m) KMP: busca padrão s em texto t, retorna
// índices de cada ocorrência
// (0-based)
vector<int> kmp(string &t, string &s) {
    vector<int> p = pi(s + '$'), match;
    for (int i = 0, j = 0; i < t.size(); ++i) {
        while (j > 0 && s[j] != t[i]) j = p[j - 1];
        if (s[j] == t[i]) j++;
        if (j == s.size()) match.push_back(i - j + 1);
    }
    return match;
}

```

7.3 Rabin-Karp

```

// O(N + M) médio, O(N * M) pior caso
// Busca de padrão p em texto t usando hashing de
// janela deslizante
// Usos:
// - Busca de múltiplos padrões simultaneamente (coloca
// todos num set de hashes)
// - Detectar strings repetidas / substrings duplicadas
// (+busca binária)

const int MOD = 1e9 + 7, BASE = 31;
vector<int> rabin_karp(string t, string p) {
    int n = t.size(), m = p.size();
    vector<int> ocorrencias;

    // Pré-computa potências de BASE
    vector<int> pw(max(n, m) + 1);
    pw[0] = 1;
    for (int i = 1; i <= max(n, m); i++) pw[i] = pw[i - 1] * BASE % MOD;

    // Hash do padrão
    int hp = 0;
    for (int i = 0; i < m; i++) hp = (hp + (p[i] - 'a' + 1) * pw[i]) % MOD;

    // Hash prefixo do texto: h[i] = hash(t[0..i-1])
    vector<int> h(n + 1, 0);
    for (int i = 0; i < n; i++) h[i + 1] = (h[i] + (t[i] - 'a' + 1) * pw[i]) % MOD;

    // Hash de t[L..r-1] = (h[r] - h[L]) / pw[L] ≡ (h[r] - h[L]) * inv(pw[L])

```

```
// Evita inversão modular: compara (h[r] - h[l])
// com hp * pw[l]
for (int i = 0; i + m <= n; i++) {
    int ht = (h[i + m] - h[i] + MOD) % MOD;
    if (ht == hp * pw[i] % MOD) ocorrencias.
        push_back(i);
}

return ocorrencias; // índices (0-based) onde p
// ocorre em t
}
```

7.4 Trie

```
// Armazena strings para buscas eficientes por prefixo
// O(N) por inserção e busca, onde N = tamanho da
// string
//
// Usos comuns em CP:
// - Contar strings com determinado prefixo
// - Verificar se string/prefixo existe
// - XOR máximo com Trie binária (ver abaixo)
//
// Como usar:
// Trie t;
// t.insert("hello");
// t.search("hello") → true (string completa existe)
// t.search("hell") → false (prefixo existe mas
// string não foi inserida)
// t.startsWith("hell") → true (prefixo existe)
// t.count("hel") → quantas strings inseridas
// começam com "hel"

struct Trie {
    struct Node {
        int filho[26]; // índice dos filhos (a=0, b=1,
        ..., z=25)
        int fim; // quantas strings terminam aqui
        int passa; // quantas strings passam por
        este nó
        Node() : fim(0), passa(0) {
            fill(filho, filho + 26, -1);
        }
    };

    vector<Node> nos;

    Trie() {
        nos.emplace_back(); // nó raiz = índice 0
    }

    void insert(const string& s) {
        int cur = 0;
        for (char c : s) {
            int ch = c - 'a';
            if (nos[cur].filho[ch] == -1) {
                nos[cur].filho[ch] = nos.size();
                nos.emplace_back();
            }
            cur = nos[cur].filho[ch];
            nos[cur].passa++;
        }
        nos[cur].fim++;
    }

    // Retorna true se a string completa foi inserida
    bool search(const string& s) {
        int cur = 0;
        for (char c : s) {
            int ch = c - 'a';
            if (nos[cur].filho[ch] == -1) return false;
            cur = nos[cur].filho[ch];
        }
        return nos[cur].fim > 0;
    }

    // Retorna true se alguma string inserida começa
    // com s
    bool startsWith(const string& s) {
        int cur = 0;
        for (char c : s) {

```

```
int ch = c - 'a';
if (nos[cur].filho[ch] == -1) return false;
cur = nos[cur].filho[ch];
}
return true;
}

// Quantas strings inseridas começam com o prefixo
// s
int count(const string& s) {
    int cur = 0;
    for (char c : s) {
        int ch = c - 'a';
        if (nos[cur].filho[ch] == -1) return 0;
        cur = nos[cur].filho[ch];
    }
    return nos[cur].passa;
}
};
```

7.5 Função Z

```
// O(n)
// z[i] = comprimento do maior prefixo de s que também
// é prefixo de s[i..]
// Usos:
// - Busca de padrão: z(p + "#" + t), ocorrências
// onde z[i] == |p|
// - Menor período de string: menor k tal que k | n e
// z[k] == n - k
// - Contar prefixos distintos que ocorrem em s:
// acumular z[i] com cuidado
// - Comparar sufixos rapidamente sem suffix array
vector<int> z_function(string s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i < r) z[i] = min(r - i, z[i - l]);

        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            z[i]++;

        if (i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

7.6 Manacher

```
// Encontra o maior palíndromo centrado em cada posição
// O(N) – linear, sem expansão quadrática
//
// Ideia central: reaproveita palíndromos já calculados
// usando a simetria do espelho em torno do centro
// atual
//
// String transformada: intercala '#' entre caracteres
// "abc" → "@#a#b#c#$"
// - '@' e '$' são sentinelas (evitam checagem de
// bounds)
// - '#' entre caracteres unifica palíndromos pares e
// ímpares
// - p[i] = raio do palíndromo centrado em i na
// string transformada
// - palíndromo real de tamanho p[i] (par ou ímpar)
//
// Exemplo: "abacaba"
// t = "@#a#b#a#c#a#b#a#$"
// p = 0 1 0 3 0 1 0 7 0 1 0 3 0 1 0
//
// Como usar:
// 1. Chame manacher(s) para pré-computar p[]
// 2. Use getLongest(i, true) → raio do maior
// palíndromo ímpar centrado em i
// Use getLongest(i, false) → raio do maior
// palíndromo par centrado entre i e i+1
// 3. O tamanho real do palíndromo é igual ao raio
// retornado
```

```
// ímpar: tamanho = 2*raio + 1, começa em i - raio
// par: tamanho = 2*raio, começa em i - raio + 1
//
// Exemplo de uso (maior palíndromo):
// manacher(s);
// for (int i = 0; i < s.size(); i++) {
//     int r = getLongest(i, true); // ímpar
//     // palíndromo: s.substr(i - r, 2*r + 1)
//     int r = getLongest(i, false); // par
//     // palíndromo: s.substr(i - r + 1, 2*r)
// }

vector<int> p;

void manacher(const string& s) {
    string t = "@";
    for (char c : s) t += "#" + string(1, c);
    t += "$";

    int n = t.size();
    p.assign(n, 0);

    int c = 0, r = 0;
    for (int i = 1; i < n - 1; i++) {
        int mirror = 2 * c - i;
        if (i < r) p[i] = min(r - i, p[mirror]);
        while (t[i + p[i] + 1] == t[i - p[i] - 1]) p[i]++;
        if (i + p[i] > r) {
            c = i;
            r = i + p[i];
        }
    }

    // ímpar: centro real em t[2*i + 2] (o próprio caractere)
    // par: centro real em t[2*i + 3] (o '#' entre i e i+1)
    int getLongest(int i, bool isOdd) {
        return p[2 * i + 2 + !isOdd];
    }
}
```

8 Árvores

8.1 LCA (Menor Ancestral Comum)

```
// Build: O(N log N) | Query: O(log N)
int LOG, n;
void dfs_lca(int u, int p) {
    up[0][u] = p;
    for (int i = 1; i <= LOG; ++i)
        up[i][u] = up[i - 1][up[i - 1][u]];

    for (auto v : tree[u]) {
        if (v != p) {
            depth[v] = depth[u] + 1;
            dfs_lca(v, u);
        }
    }
}

int lca(int u, int v) {
    if (depth[u] < depth[v]) swap(u, v);
    int diff = depth[u] - depth[v];

    for (int i = 0; i <= LOG; ++i)
        if (diff & (1 << i)) u = up[i][u];

    if (u == v) return u;

    for (int i = LOG; i >= 0; --i) {
        if (up[i][u] != up[i][v]) {
            u = up[i][u];
            v = up[i][v];
        }
    }

    return up[0][u];
}

void build_lca(int root = 0) {
```

```
    LOG = (int)ceil(log2(n));
    depth.assign(n + 1, 0); // 1-based
    up.assign(LOG + 1, vector<int>(n + 1));
    tree.assign(n + 1, {});
    dfs_lca(root, root);
}

// Distância entre dois vértices
int dist_lca(int u, int v) {
    return depth[u] + depth[v] - 2 * depth[lca(u, v)];
}

// k-ésimo ancestral (k=0: próprio nó, k=1: pai)
int kth_ancestor(int u, int k) {
    for (int i = 0; i <= LOG; i++)
        if (k & (1 << i)) u = up[i][u];
    return u;
}
```

8.2 Centroid Decomposition

```
// O(N)
int n;
vector<vector<int>> tree;
// 1-based, subtree inclui o nó raiz = inicializa com 1
vector<int> subsize(n + 1, 1);
void dfs(int u, int p) {
    for (auto& v : tree[u])
        if (v != p) {
            dfs(v, u);
            subsize[u] += subsize[v];
        }
}

int get_centroid(int u, int p) {
    for (auto& v : tree[u]) {
        if (v == p) continue;
        if (subsize[v] * 2 > n) return get_centroid(v, u);
    }
    return u;
}
```

9 Geometria

9.1 Operações básicas

```
// === 2D Geometry Basics ===
// Complexidade: O(1) para todas operações básicas

using ld = long double;
const ld EPS = 1e-9;
const ld PI = acos(-1);

struct Point {
    ld x, y;
    Point(ld x = 0, ld y = 0) : x(x), y(y) {}

    Point operator+(Point p) { return Point(x + p.x, y + p.y); }
    Point operator-(Point p) { return Point(x - p.x, y - p.y); }
    Point operator*(ld d) { return Point(x * d, y * d); }
    Point operator/(ld d) { return Point(x / d, y / d); }

    bool operator<(Point p) const {
        if (fabs(x - p.x) > EPS) return x < p.x;
        return y < p.y - EPS;
    }
    bool operator==(Point p) { return fabs(x - p.x) < EPS
        && fabs(y - p.y) < EPS; }

    ld dist2() { return x * x + y * y; }
    ld dist() { return sqrt(dist2()); }
    ld dot(Point p) { return x * p.x + y * p.y; }
    ld cross(Point p) { return x * p.y - y * p.x; }
    ld angle() { return atan2(y, x); }
    Point rotate(ld a) { return Point(x*cos(a) - y*sin(a),
        x*sin(a) + y*cos(a)); }
    Point perp() { return Point(-y, x); } // rotaciona 90° CCW
    Point unit() { return *this / dist(); }
}
```

```

};

// Distância entre pontos
ld dist(Point a, Point b) { return (a - b).dist(); }

// Ângulo entre vetores ( $\theta$  a  $PI$ )
ld angle(Point a, Point b) { return acos(a.dot(b) / (a.
    dist() * b.dist())); }

// Orientação:  $\theta =$  colinear,  $>\theta =$  CCW,  $<\theta =$  CW
ld orient(Point a, Point b, Point c) { return (b - a).
    cross(c - a); }

// Projeção de p na reta ab
Point project(Point a, Point b, Point p) {
    Point v = b - a;
    return a + v * ((p - a).dot(v) / v.dist2());
}

// Distância de ponto p à reta ab
ld dist_to_line(Point a, Point b, Point p) {
    return fabs((p - a).cross(b - a)) / (b - a).dist();
}

// Distância de ponto p ao segmento ab
ld dist_to_seg(Point a, Point b, Point p) {
    if (a == b) return dist(a, p);
    Point v = b - a;
    ld t = (p - a).dot(v) / v.dist2();
    if (t < 0) return dist(a, p);
    if (t > 1) return dist(b, p);
    return dist_to_line(a, b, p);
}

// Interseção de retas ab e cd (assume não paralelas)
Point line_intersect(Point a, Point b, Point c, Point d
    ) {
    Point v1 = b - a, v2 = d - c;
    ld t = (c - a).cross(v2) / v1.cross(v2);
    return a + v1 * t;
}

// Segmentos ab e cd se intersectam?
bool seg_intersect(Point a, Point b, Point c, Point d)
{
    ld o1 = orient(a, b, c), o2 = orient(a, b, d);
    ld o3 = orient(c, d, a), o4 = orient(c, d, b);
    if (o1 == 0 && o2 == 0 && o3 == 0 && o4 == 0) {
        if (b < a) swap(a, b); if (d < c) swap(c, d);
        return !(b < c || d < a);
    }
    return ((o1 > EPS) != (o2 > EPS)) && ((o3 > EPS) != (
        o4 > EPS));
}

```

9.2 Fecho convexo

```

// == Andrew's Monotone Chain ==
// Complexidade:  $O(N * \log N)$ 
// Retorna pontos do casco em ordem CCW, sem pontos
// colineares na borda

vector<Point> convex_hull(vector<Point> pts) {
    if (pts.size() <= 1) return pts;
    sort(pts.begin(), pts.end());
    vector<Point> hull(pts.size() * 2);
    int k = 0;

    // Lower hull
    for (int i = 0; i < pts.size(); i++) {
        while (k >= 2 && orient(hull[k-2], hull[k-1], pts[i])
            <= EPS) k--;
        hull[k++] = pts[i];
    }

    // Upper hull
    for (int i = pts.size()-2, t = k+1; i >= 0; i--) {
        while (k >= t && orient(hull[k-2], hull[k-1], pts[i])
            <= EPS) k--;
        hull[k++] = pts[i];
    }
}

```

```

hull.resize(k-1);
return hull;
}

// Área do polígono (pontos em ordem CCW ou CW)
// Complexidade:  $O(N)$ 
ld polygon_area(vector<Point> &poly) {
    ld area = 0;
    int n = poly.size();
    for (int i = 0; i < n; i++)
        area += poly[i].cross(poly[(i+1) % n]);
    return fabs(area) / 2.0;
}

// Perímetro do polígono
// Complexidade:  $O(N)$ 
ld polygon_perimeter(vector<Point> &poly) {
    ld per = 0;
    int n = poly.size();
    for (int i = 0; i < n; i++)
        per += dist(poly[i], poly[(i+1) % n]);
    return per;
}

// Ponto dentro do polígono? (assume polígono simples)
//  $\theta =$  fora, 1 = dentro, 2 = na borda
// Complexidade:  $O(N)$ 
int point_in_polygon(vector<Point> &poly, Point p) {
    int n = poly.size(), wn = 0;
    for (int i = 0; i < n; i++) {
        Point a = poly[i], b = poly[(i+1) % n];
        if (dist_to_seg(a, b, p) < EPS) return 2; // na
        borda
        if (a.y <= p.y + EPS) {
            if (b.y > p.y + EPS && orient(a, b, p) > EPS) wn
                ++;
        } else {
            if (b.y <= p.y + EPS && orient(a, b, p) < -EPS)
                wn--;
        }
    }
    return wn != 0;
}

```

10 Algoritmos de Ordenação

10.1 Insertion Sort

```

//  $O(N^2)$ 
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; ++i) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

```

10.2 Merge Sort

```

//  $O(N * \log N)$ 
void merge(vector<int>& vet, int left, int mid, int
    right) {
    vector<int> temp(right - left + 1);
    int i = left, j = mid + 1, k = 0;
    while (i <= mid && j <= right) {
        if (vet[i] <= vet[j])
            temp[k++] = vet[i++];
        else
            temp[k++] = vet[j++];
    }
    while (i <= mid)
        temp[k++] = vet[i++];
    while (j <= right)
        temp[k++] = vet[j++];
    for (i = left, k = 0; i <= right; i++, k++)
        vet[i] = temp[k];
}

```

```
// Chamada => merge_sort(vet, 0, vet.size() - 1);
void merge_sort(vector<int>& vet, int left, int right)
{
    if (left < right) {
        int mid = left + (right - left) / 2;
        merge_sort(vet, left, mid);
        merge_sort(vet, mid + 1, right);
        merge(vet, left, mid, right);
    }
}
```

10.3 Quicksort

```
// O(N * LogN)
int partition(vector<int>& vet, int left, int right) {
    int mid = left + (right - left) / 2;
    int pivot = vet[mid];
    swap(vet[mid], vet[right]);
    int i = left - 1;
    for (int j = left; j < right; j++) {
        if (vet[j] <= pivot) {
            i++;
            swap(vet[i], vet[j]);
        }
    }
    swap(vet[i + 1], vet[right]);
    return i + 1;
}
```

```
// Chamada => quick_sort(v, 0, v.size() - 1);
void quick_sort(vector<int>& vet, int left, int right)
{
    if (left < right) {
        int pivot = partition(vet, left, right);
        quick_sort(vet, left, pivot - 1);
        quick_sort(vet, pivot + 1, right);
    }
}
```

10.4 Counting Sort

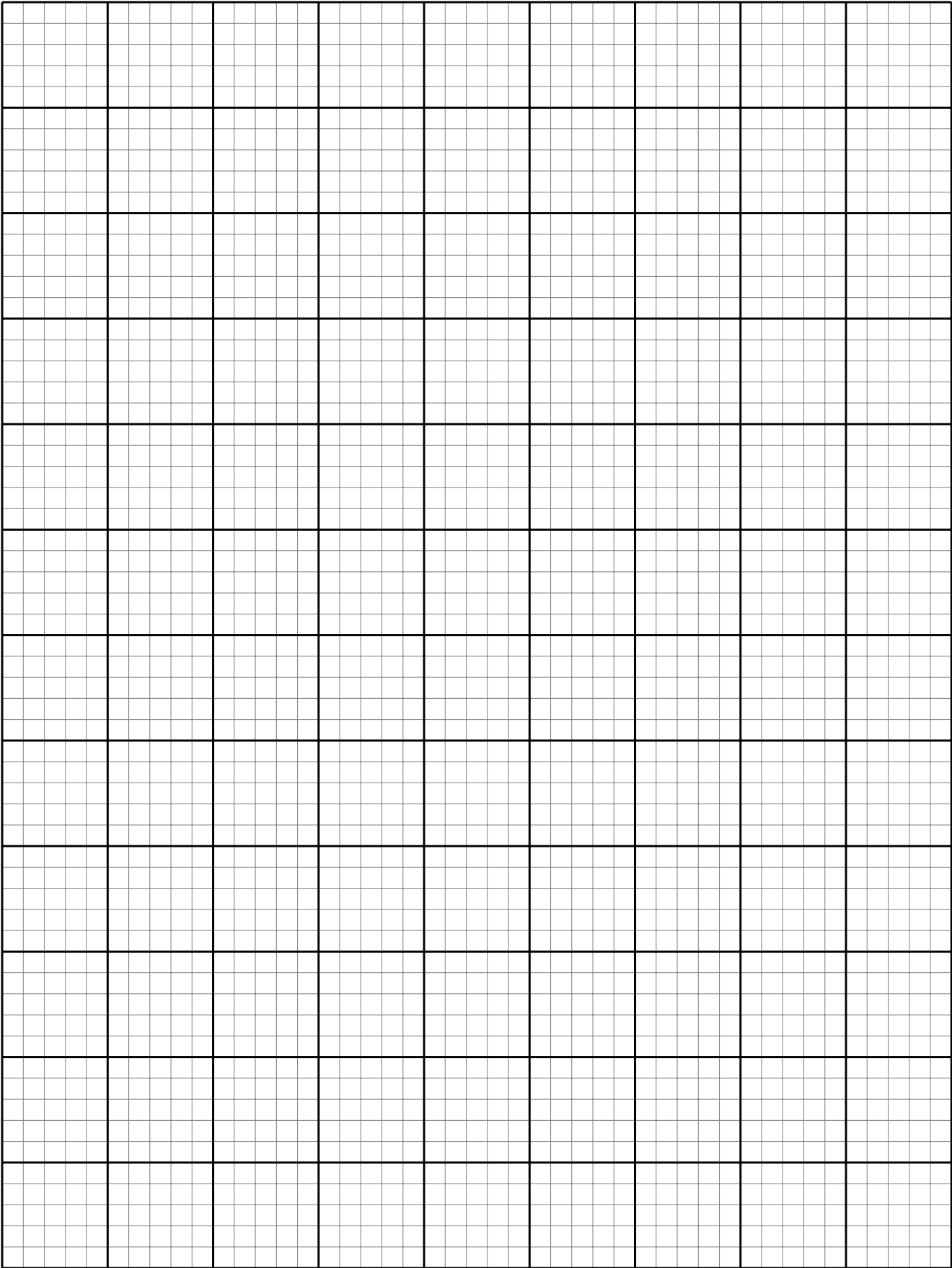
```
// O(N + K), K = max_val
void counting_sort(vector<int>& vet, int max_val) {
    vector<int> count(max_val + 1, 0);
    for (int x : vet) count[x]++;
    int idx = 0;
    for (int i = 0; i <= max_val; i++)
        while (count[i]--) vet[idx++] = i;
}
```

11 Problemas Especiais

11.1 Torre de Hanoi

```
// Torre de Hanoi: move n discos de 'a' para 'b' usando
// 'c' como auxiliar
// Complexidade: O(2^n), gera exatamente 2^n - 1
// movimentos
vector<pair<int, int>> moves;
void hanoi(int n, int a, int b, int c) {
    if (n == 1) moves.push_back({a, b});
    else {
        hanoi(n - 1, a, c, b);
        hanoi(1, a, b, c);
        hanoi(n - 1, c, b, a);
    }
}
// Uso: hanoi(n, 1, 3, 2);
```


Papel milimetrado para rascunho



[illegible]